Date: 05/04/2026

Chai Aur SQL Part-2

Why we save phone number is varchar?

phone-number VARCHAR(15) UNIQUE,

- If you use INT instead of VARCHAR
- INT in postgres is 4byte integer
  = 32 = 2^32 = 4 Billion (possible value we can store)
- But this is signed integer
  that's mean you can have -ve as well as +ve
  - 2 billion will be -ve
  - 2 billion will be +ve

let's understand with indian phone number
ex,
9999999999 if it's integer = 9.9 Billion (possible value
and it's greter than the 2 billion +ve

So, technically we can't use INT for phone-number

But what if we store in BIGINT,
because it's allows to store very large numbers.

- BIGINT can store upto 8 bytes = 8 * 8 bits = 64 bits
  = 2^64 = 9 with 12 zeros

1 byte = 8 bits

So, technically we can store the value in BIGINT
i mean numbers.

But only when you know your number is digit

Date: / /

ex,

if you have phone number like this

0987654321 the BIGINT converts it into
⇒ 987654321 (it removes the zero)

- In integer we can not lead zero
- so, decimals do not have leading zeros

and, some countries use '-' in their numbers
ex,:

987-654321
- this is not a valid integer

- That's why we use VARCHAR and 15 for safer side.

Note:- INT ⇒ 4 bytes vs BIGINT ⇒ 8 bytes vs VARCHAR ⇒ 10 bytes

- But because of business requirements, I might have number with hyphen '-' and leading zero

-- Joining

It's simply means ~~combining~~ combining two tables
But we need a surface of joining
ex,
When we joined the wood block we need a one strip

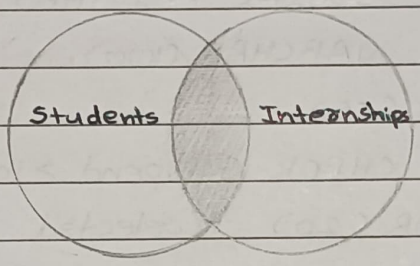
Foreign ⇒ Bahar desh ka

Date: / /

But as soon as it connects, the entire strip becomes common.

Type of joins in SQL
- LEFT JOIN
- RIGHT JOIN
- INNER JOIN
- FULL OUTER JOIN

1) INNR JOIN



- It only contain rows, jo dono table ko satisfy karti hai
- Students or Internship ke bich ka jo common part hai agar vo fullfill ho raha hai to vahi row usme aayegi
- This will only return student's who got the intership and it will only return internships which are gotten by students

Foreign Key:-

serial ⇒ int ~~and~~ without increemment
- 1st table ki primary key kisi or table me store karna matlab hi foreign key
- ex,

Jo bhi mera table hai usme agar koi column hai, jiski value kisi or table ko donate karti hai vo foreign key

Note:- Serial under the hood int hai.

Date: / /

~~01-left-join.sql~~ 01-inner-join.sql

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR (100),
    email VARCHAR (100),
    branch VARCHAR (50)
);

CREATE TABLE internships (
    internships_id SERIAL PRIMARY KEY,
    company_name VARCHAR (100),
    role VARCHAR (50),
    stipend INT CHECK (stipend > 10000),
    status VARCHAR (20) -- selected / pending / Rejected
    student_id INT REFERENCES students (student_id)
)
```

REFERENCES

- student_id is referencesing my students table ka student_id column
- student_id is a foreign key inside internship which is pointing to students table.

I have two tables students & Internships

If you might have operation, jaha pe mujhe internship me get api lena hai and also want ki har ek column me kon konse user hai kon kon students hai so.

I can use joines to actually fetch the students detail based on students_id

Date: / /

Student_id is foreign key of Internship

If student delete their details the internship also got delete

ON DELETE

Student_id INT REFERENCES students (student_id)
ON DELETE
CASCADE

- agar student_id ex. 4 agar delete hota hai to ON DELETE then I want to CASCADE.
- jis bhi table me student_id ka reference hoga agar vo on DELETE CASCADE hai to uske related jitne bhi rows hai ke bhi table me use delete kar do.

ON DELETE you cant not only CASCADE, you can set as null

Student_id INT REFERENCES Students (student_id)
ON DELETE SET NULL

- If student delete his account then studen_id in othe table is going to be null
- This is used for soft deletation
- Internship ka data rahega par student_id null ho jayega
- data hai par student delete hai.

Date: / /

RESTRICT

Student_id INT REFERENCES students (student_id)
ON DELETE RESTRICT

- agr mere kisi bhi student ko internship laga hai, fir vo student delete nahi ho sakta hai

Note:-
- Database dosn't care of data.
- once you delete the student_id data example x student,
- then that x student doesn't exist in db but IF that x student come again then her id is never be same
- for db x student is new student
- even if human is same.
But id is different whatever next id it is
- database doesn't care you delete or not, it's always gives next id.
- this is USP (Cuser defined stored procedure) in sql

foreign key is the key which is coming from anothe table

Inener join

common, jin students ko internship mili hai bas vahi bagenge, inner join me.

- In the process of Inner join you can also lose the data

multijoin (This is the wrong of doing multiple joinning)
But Kuchh kase me hi ye Karna hota hai,
mostly primary key pe hi join lagaya jata hai.

```
SELECT
    s.*,
    i.company_name
FROM students AS s
INNER JOIN internships AS i ON (i.student_id = s.student_id AND i.company_name = s.name);
```

Date: / /

```
SELECT
    students.name,
    students.branch,
    internships.company_name,
    internships.role,
    internships.stripend
FROM students
INNER JOIN internships ON
    students.student_id = internships.student_id
```

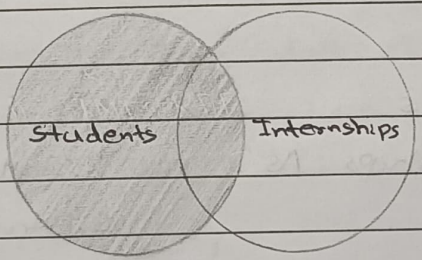
+ Wahi student table me display honge jinhe internship mili hai

If you want all the tables then you can use *

s.* (students table)

i.* (internship table)

LEFT JOIN



+ student table ka sara data aayega par internship ka wahi data aayega jo student ke sat combine ho raha hai

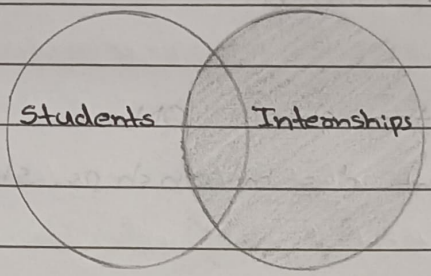
02-left-join.sql

```
SELECT
    s.name,
    s.branch,
    i.company_name,
    i.stripend
```

Date: / /

```
FROM students AS s
LEFT JOIN internships AS i ON
        i.student_id = s.student_id;
```

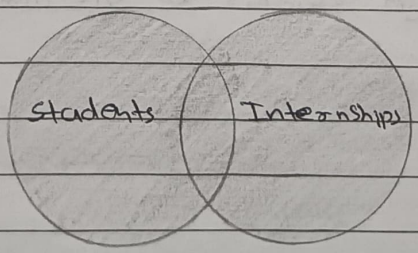
REGHT join



- isme internships table ka sara data hoga, pair students table ka vahi data hoga jo internships table se combine ho raha hai

```
SELECT
    s.name,
    s.branch,
    i.company_name,
    i.stripend
FROM students AS s
RIGHT JOIN internships AS i ON i.student_id =
                        s.student_id;
```

Full-outer join



- It gives all the data from both table,
- jo data nahi hai usko add kar dega or use null kar dega.

Date: / /

03-full-outer-join.sql

```
SELECT
    s.name AS student_name,
    s.branch,
    i.company_name,
    i.status
FROM students s
FULL OUTER JOIN internships i ON s.student_id =
                                 i.student_id;
```

Summary

1) Inner join :- do table me jo condition fulfill kar raha hai vahi data aaye ge

2) left join :- left join ke left table ka sara data aayega par internships ka vahi data aayega jiski condition true hai

3) full outer join :- sara data dono table ka aayega

Note :-

Don't use right join.

Indexes

How to analyse the query

In Dictionary if you wanted to find the meaning of mat, what you do.

you simply go to index page then search for m then go direct to that page number.

Date: / /

- that is indexes. db's index also work the same.
- DB index actually tells DB ki jo data aapko chahiye vh memory ke kis kone mein hai
- Indexs can be done on any key.
- you can also do ~~index~~ query without ~~indexes~~. too.

EXPLAIN ANALYZE

explain analyze ye kya karta hai,
ye jo query hai vo kitna time legi or kya kya tegi karegi
This is your jumping into db internal
and this is exactly what your db engine does.
execution time kitna lagega ye bhi batata hai

Parallel seq Scan

let say,
you have 1 million rows
so, what parallel seq scan do, when you say
name = "xyz"
then he started to check. 1, 2, 3 ... rows
until he finds the match
It's parallel
because prostgress internally spins the workers
It simply means, It's scan sequentially

```
SELECT * FROM marks;

EXPLAIN ANALYZE SELECT * FROM marks
        WHERE name = '5178EE8BF887';
CREATE INDEX idx-name ON marks (name);
```

Note:- Drawback of indexes, it's takes time to create.

Date: / /

CONCURRENTLY

- When this option is used, PostgreSQL will build the index without taking any locks that prevent concurrent inserts, updates or deletes on the table

- that's mean it's create inserts but it does not lock the table.
that's mean if i create table, it's allow me to insert.

And this decision for DB admin not for developers to do.

- and this decision should be taken intially

- Index store on disk.

Index Scan

```
SELECT * FROM marks WHERE name = '5178FE8BF887'
```

Index		Actual table
5178FE8BF887 16654321 memory is location pe abc 111 loc pqr 121 loc		id:1, name:abc, marks:10 id:2, name:pqr, marks:10, id:3, name:xyz, marks:10, ... id:1000000, name:abc, marks:10,

Date: / /

- When you run the query without index tab aap table ke har ek row pe jake id check karoge, agar Id last me hai to aap ko million time bhi for loop chal sakta hai

- par Index me,
prostgres actual table ke pass jane se pehle index ke pass jayegen.
or ye sab B+ tree me hota hai.
isme login log of me hota hai, matlab har row ko ek bar check karegen.

- par index ka look up jo hota hai, vo B+ tree me log n pe hota ha.
B+ tree are not binary trees, those are balance tree.
and it's use mostly in indexing

- So, when you creat indexe in prostgres it's actually creat a B+ tree indxes.

- matlab index me value search karna actual table me search karne se bhi aasan hai
- Index scan is very fast.

- So, using index scan you can get directly the location in memory, you and directly get data from actual table.

- Table ek ke niche ek store hai
- par Index tree structure me store hai
- isliye Indexes me searching fast hai

Note:- As a developer you just create index other things DB can do like query plan

Date: / /

TRANSACTION

It's a operation in DB, jo bhi query aap chalao ge in the transaction wo either completely ~~sucs~~ success hoge ~~nahi~~ to as a group nahi to completly fail hoga as a group their is not a success and failure

Partial

ex,

upi transaction

name	amount
Shubham sir	5000
Hitesh sir	5000

TXN 1

```
BEGIN;

UPDATE accounts SET amount = amount -
    500 WHERE name = 'Shubham sir';

Time

UPDATE accounts SET amount = amount +
    500 WHERE name = 'Hitesh sir';

Commit :- This will commit the changes
ROLLBACK :- This will revert the changes
    done after BEGIN Keyword
```

- Commit ke bad ROllback nahi hota hai
- you BEGIN you do something, you commit the rollback that's all transaction

Note: For db transaction any failure is rollback default action is rollback.
unless you explecitly says ki Comimit karna hai db rollback karta hai.

Date: / /

ACID

A ⇒ Atomicity
C ⇒ Consistency
I ⇒ Isolation
D ⇒ Durability

Atomicity

agar 100 query of transaction execute ho rahi hai. ek to 100 ki 100 query succesfull hogi nahi to 100-100 query fail hogi
100% or 0% thats the atomicity

Isolation

mere first transaction me jo changes kiye hai. vo mere second transaction ko pata ni nahi hoge, jab tak first transaction commit nahi ho jata that is isolation
mere 1st transaction ka update mere second transaction ko pata nahi chal raha hai.

HomeworK

dirty read :- In SQL is a concurrency problem that occurs when a transaction reads data that has been modified by another concurrent transaction but has not yet been commited.

Date: / /

phatoms :- This is a problem occurs within a transaction when the same query produces different sets of rows at different times.

ex.

if a SELECT is executed twice, but returns a row the second time that was not returned the first time, the row is a Phantom row

Non-Repeatable Read :- Reading the same row twice and getting different values.

Consistency

Because of atomicity and isolation your data will be consistency

- Data should not break your expectations

Durability

If I have my disk and If I'm commiting it better have data in my disk.

- agr db ne kaha ke ab me commit kiya hai to data mast be there

ACID compliance in database

The set of four key properties - Atomicity, Consistency, Isolation, Durability

ex.

Postgres / MySQL / Oracle DB / Couchbase / ~~Redis~~

Which DB is not acid compliances - Apache Cassandra

Note: Redis is in memory db
Redis store data in RAM - and it's really fast

Date: / /

Database Design (Instagram)

// Users, posts, story, comments, likes, followers, bookmarks

Note

Difference between CHAR & VARCHAR

If

CHAR(10)

VARCHAR(10)

But what db does

In char it will leave ten spaces.

and the you add ABC string

then it's

CHAR(10) ⇒ 'ABC-------''ABC'

But in VARCHAR it dosn't leve spaces.

VARCHAR(10) ⇒ 'ABC'

VARCHAR ⇒ Variable charecter

Users [icon: user] {

id serial PK

username varchar (50) unique not null

email varchar(100) unique not null

phone char(15)

dob date

password varchar (256)

bio text

avatar_url text

forgot_password_token text

email_verification_token text

created_at timestamp

updated_at timestamp

}

relation :- one to one (-)
one to many (<)
many to one (>)

Date: / /

```
Posts [icon: notbook] {
  id serial PK
  Caption text
  image-url text null // can be null
  video_ url text null // can be null
  reel_ url text null // can be null
  auther-id int FK
  created_at timestamp
  updated_at timestamp
}

users.id < Posts.auther_id

Stories [icon: notbook] {
  id serial PK
  image-url text null // can be null
  video_ url text null // can be null
  post_id int fk null // can be null
  auther_id int FK
  created_at timestamp
  updated_at timestamp
}

user.id < stories.auther_id
Stories.post_id - Posts.id

bookmarks [icon: bookmark] {
  id serial PK
  post_id int fk
  auther_id int FK
  created_at timestamp
  updated_at timestamp
}
```

Date: / /

```
user.id < bookmarks.auther_id
bookmarks.post_id - posts.id

account_setting [icon:account_setting] {
    id serial PK
    user_id int FK
    is_account_private boolean
    created_at timestamp
    updated_at timestamp
}

user.id - account_setting.id

Comments [icon: message_circle] {
    id serial PK
    auther_id int FK
    post_id int FK
    content text
    created_at timestamp
    updated_at timestamp
}

Comments.auther_id > users.id
comments.post_id > posts.id

likes [icon: heart] {
    id serial PK
    auther_id int FK
    post_id int FK
    created_at timestamp
    updated_at timestamp
}
```

Date: / /

likes.auther-id > users.id

likes.post-id > posts.id

followers [icon: connection] {

id serial PK

user-id int FK

created-at timestamp

updated-at timestamp

}

users.id <> followers.id